

37.2 Dream --- техническая спецификация

Рабочее название: 37.2 Dream\ Репозиторий: 37-2-dream\ Тип: процедурный browser-based surreal dream experience / game\ Платформы: Desktop Web, Mobile Web, PWA\ Язык документа: русский\
Версия: 1.1

1. Концепция

37.2 Dream --- браузерный процедурно генерируемый сон мужчины с температурой 37,2 °C.

Главный принцип: игра не должна выглядеть как набор случайных комнат. Она должна создавать ощущение цельного, но абсолютно нелогичного сна, где каждый запуск отличается последовательностью, комбинацией, физикой, звуком и событиями.

Игрок не получает классическую цель «победить врага». Основная задача --- исследовать сон и посмотреть, насколько далеко он способен зайти.

Ключевой юмористический принцип:

Всё происходящее подаётся максимально серьёзно, хотя логика происходящего полностью абсурдна.

Повторяющиеся мотивы создают узнаваемый язык проекта:

- температура 37,2 °C;
- пустой советский двор;
- скрипучие качели;
- женщина с пылесосом;
- медицинский монитор;
- звук ИВЛ;
- бесконечные лестницы без перил;
- падение;
- варенье;
- голуби;
- завещание;
- лифт;
- термометры;
- медицинские и бытовые звуки;
- внезапные аудио- и видеоскриммеры.

2. Главная функциональная идея: уникальный сон

Каждый запуск должен создавать новый сон.

Допускается повторение отдельных локаций и объектов, но полная последовательность и комбинация элементов должна быть крайне маловероятной к повторению.

Нужно избегать:

- простого random() для выбора следующей комнаты;

- заранее заданного линейного сценария;
- полного рандома без драматургии;
- повторения одного и того же набора событий.

Используется seeded procedural dream generation.

Каждый сон получает уникальный DreamSeed.

Пример:

DREAM-8F72-A19C-37B2

Seed определяет:

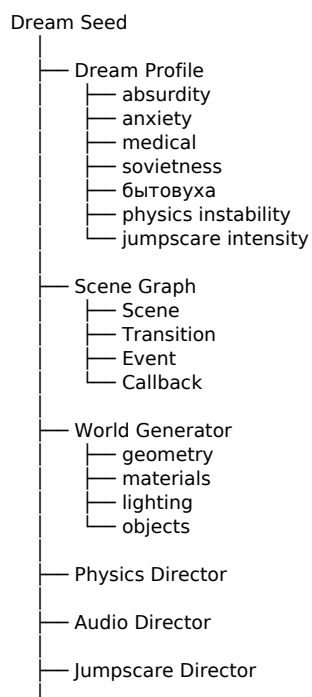
- последовательность сцен;
- набор локаций;
- физические параметры;
- комбинации объектов;
- переходы;
- аудио;
- скриммеры;
- диалоги;
- интенсивность абсурда;
- recurring jokes;
- продолжительность;
- вероятность повторного появления ранее увиденных элементов.

При одинаковом seed сон должен воспроизводиться детерминированно.

При новом seed --- создавать другую комбинацию.

3. Архитектура генератора сна

Генератор должен быть многоуровневым.



4. Dream Profile

Перед генерацией создаётся профиль сна.

Пример:

```
{
  "temperature": 37.2,
  "absurdity": 0.91,
  "anxiety": 0.58,
  "medicalIntensity": 0.67,
  "sovietIntensity": 0.74,
  "domesticIntensity": 0.81,
  "physicsInstability": 0.63,
  "jumpscareIntensity": 0.42,
  "dreamLength": 8.7
}
```

Параметры должны не только выбирать контент, но и влиять на его поведение.

Например:

- высокий `medicalIntensity` → больше термометров, мониторов, ИВЛ и

пиканья;

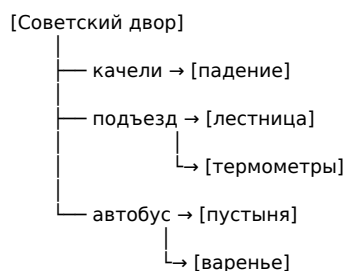
- высокий `sovietIntensity` → дворы, подъезды, панельные дома, качели;
- высокий `physicsInstability` → инверсия гравитации, лестницы,

падения;

- высокий `absurdity` → более невозможные комбинации;
- высокий `jumpscareIntensity` → больше ложных и настоящих скриммеров.

5. Scene Graph вместо линейного сценария

Сон представляется графом.



Но конкретный путь определяется процедурно.

Например, один сон:

```

Двор
→ Качели
→ Падение
→ Лестница
→ Лифт
→ Море
→ Варенье
→ Пылесос
→ Коридор термометров
→ Пробуждение
  
```

Другой:

```

Квартира
→ Холодильник
→ Лестница
→ Советский двор
→ Голубь
  
```

- Автобус 37,2
- Небо
- Падение
- Завещание
- Пылесос
- Двор

Третий:

- Пустыня
- Качели
- Лифт
- Больничный коридор
- Варенье
- Подводная физика
- Термометры
- Ложный скриммер
- Настоящий скриммер
- Спальня

6. Контентные модули

Каждая сцена состоит из независимых модулей.

6.1 Locations

Базовый набор:

1. Советский двор.
2. Советский подъезд.
3. Бесконечная лестница без перил.
4. Квартира.
5. Спальня.
6. Кухня.
7. Лифт.
8. Пустыня.
9. Гигантская банка варенья.
10. Коридор из термометров.
11. Больничный коридор.
12. Небо.
13. Море.
14. Автобус.
15. Панельный город.
16. Комната с дождём.
17. Магазин.
18. Невозможная квартира.
19. Огромный холодильник.
20. Пустая детская площадка.

7. Советский двор

Центральная recurring-локация.

Компоненты:

- панельный дом;
- мокрый асфальт;
- жёлтые окна;
- фонари;
- мусорные контейнеры;
- детская площадка;
- качели;
- лавочки;
- голуби;
- припаркованные автомобили;
- редкие окна с движением штор.

Качели

Качели должны быть отдельным procedural event.

Состояния:

```
idle
slow_swing
fast_swing
empty_swing
player_swing
impossible_swing
transition
```

Ключевой звук:

```
скрип...
скрип...
```

При достижении определённой амплитуды мир может:

- масштабироваться;
- менять гравитацию;
- менять время суток;
- заменять фон;
- превращать двор в пустыню;
- открывать портал;
- запускать падение.

8. Гигантская банка малинового варенья

Игрок попадает в гигантскую банку.

Физика становится похожей на движение под водой.

Среда:

- вязкость;
- пузырьки;
- полупрозрачность;
- частицы;
- лимоны размером с метеориты;

- малиновые косточки;
- гигантские ложки;
- этикетки;
- рыбки.

Периодически направление гравитации меняется.

Главное правило:

Варенье должно выглядеть почти физически правдоподобно, пока происходящее не начинает противоречить здравому смыслу.

9. Женщина с пылесосом

Recurring character/event.

Она появляется:

- во дворе;
- в пустыне;
- в больничном коридоре;
- на крыше;
- в падении;
- внутри банки варенья;
- на Луне;
- в автобусе;
- в зеркале.

Она не объясняется.

Она просто пылесосит.

Аудио

Сначала:

вжж...

Затем:

вжжжжжжж...

Затем звук может трансформироваться в:

- турбину;
- реактивный двигатель;
- гул самолёта;
- промышленную вентиляцию.

На мобильном устройстве при поддержке API допускается краткая вибрация.

10. ИВЛ и медицинские звуки

Медицинские звуки являются редкими recurring motifs, а не постоянным саундтреком.

ИВЛ

Тихий звук:

ф-ф-ф...
шшш...
ф-ф-ф...

Использование:

- перед сменой реальности;
- во время длительного падения;
- в медицинском коридоре;
- во время погружения в варенье;
- перед некоторыми скриммерами;
- перед пробуждением.

Иногда источник должен быть визуально обнаружен.

Например:

посреди советского двора стоит медицинский аппарат ИВЛ.

Он работает.

Никого рядом нет.

Через несколько секунд двор становится обычным.

11. Реанимационный монитор

Звук:

пип...
пип...
пип...

Интенсивность может динамически изменяться.

Во время опасного события:

пип... пип... пип-пип... пип-пип-пип...

Перед критическим переходом:

пиппиппиппип

После перехода:

тишина.

Затем:

пип.

И новая сцена.

12. Коридор из термометров

Стены состоят из гигантских термометров.

Показания:

37.1
37.2
37.3
37.2
37.1

Возможные варианты:

- 36.9;
- 37.0;
- 37.2;
- 37.7;
- 39.0;
- 42.0.

Чем выше температура, тем нестабильнее мир.

Дверь:

НЕ ОТКРЫВАТЬ ПРИ ТЕМПЕРАТУРЕ НИЖЕ 37,2

За дверью может оказаться:

- тот же коридор;
- советский двор;
- спальня;
- море;
- варенье;
- бесконечная лестница.

13. Падение

Падение --- один из основных типов перехода.

Во время падения вокруг могут пролетать:

- диваны;
- холодильники;
- автобусы;
- лестницы;
- голуби;
- термометры;
- пылесосы;
- кровати;
- куски зданий;
- туалетная бумага;
- документы.

Падение не обязательно заканчивается смертью.

Возможные исходы:

- удар о землю → новая сцена;
- падение в воду;
- падение в варенье;
- падение в кровать;
- падение в лифт;
- падение вверх;

- бесконечное падение;
- просыпание.

14. Завещание

Во время падения пролетают:

- салфетки;
- туалетная бумага;
- квитанции;
- документы.

На них постепенно появляется текст.

Темы:

- кот;
- квартира;
- Wi-Fi;
- телевизионный пульт;
- холодильник;
- пылесос;
- варенье.

Стиль текста нарочито официальный и бюрократический.

15. Лестница

Лестницы должны процедурно изменяться.

Варианты:

- вверх;
- вниз;
- горизонтально;
- вертикально;
- по потолку;
- по стене;
- бесконечная;
- с исчезающими ступенями;
- с разными номерами этажей.

Особый вариант:

37
37.1
37.2
37.3
38
37.2
-37
0
37.2

16. Скриммер-система

Скриммеры делятся на:

1. Visual.
2. Audio.
3. Video.
4. False scare.
5. Delayed scare.
6. Environmental scare.

Важное правило

Скриммер нельзя выбирать полностью случайно.

У него должны быть:

Trigger
Anticipation
Misdirection
Scare
Recovery
Cooldown

Пример:

пустой коридор
→ пиканье
→ открывающийся лифт
→ ожидание монстра
→ голубь
→ расслабление
→ через 3 секунды настоящий скриммер

17. Видео-скриммеры

Допускаются короткие pre-rendered video clips.

Рекомендуемая продолжительность:

- 100--300 мс --- flash scare;
- 300--800 мс --- быстрый visual scare;
- 1--2 секунды --- cinematic scare.

Темы:

- лицо;
- силуэт;
- человек в окне;
- человек на качелях;
- гигантский голубь;
- медицинский монитор;
- женщина с пылесосом;
- собственная кровать;
- собственное лицо.

Видео не должно постоянно повторяться.

Используется cooldown и история уже показанных scare events.

18. Уникальность сна

Нужны несколько уровней защиты от повторов.

18.1 Global Seed

Уникальный seed всего сна.

18.2 Scene Seed

Каждая сцена получает собственный seed:

`DreamSeed + SceneIndex`

18.3 Event Seed

Каждое событие:

`DreamSeed + SceneIndex + EventIndex`

18.4 History

Хранить локально:

```
recentScenes
recentTransitions
recentJumpscare
recentAudioEvents
recentCharacters
recentObjects
```

Недавние комбинации получают штраф вероятности.

18.5 Weighted rarity

Каждый элемент имеет:

```
baseWeight
rarity
cooldown
compatibility
incompatibility
```

Пример:

```
{
  "id": "woman_vacuum",
  "baseWeight": 0.3,
  "rarity": 0.7,
  "cooldownScenes": 3
}
```

19. Compatibility Matrix

Не все элементы должны сочетаться.

Пример:

Element A Element B Result -----
Soviet yard Swings +++ Soviet yard Vacuum
woman ++ Jelly Lemon meteorites +++ Hospital Ventilator +++ Desert Hospital bed ++ Elevator Ocean
+++ Stairs Gravity inversion +++ Swings Jelly \+ Hospital Soviet yard ++ Bus Sky +++

Это позволит создавать контролируемый абсурд.

20. Температура как глобальный параметр

37,2 °C --- не просто текст.

Она является состоянием сна.

temperatureState = 37.2

Температура может колебаться:

37.1 → 37.2 → 37.3 → 37.2

При повышении:

- растёт chromatic aberration;
- увеличивается distortion;
- усиливается звук;
- появляются ошибки физики;
- увеличивается вероятность скриммера;
- объекты начинают масштабироваться;
- переходы становятся быстрее.

При снижении:

- мир становится спокойнее;
- звуки приглушаются;
- камера стабилизируется.

21. Аудио-архитектура

Аудио разделить на слои:

Ambient
↓
Location
↓
Physics
↓
Recurring
↓
Tension
↓
Scare
↓
Transition

Одновременно активны только совместимые слои.

Основные recurring sounds

- качели;
- ИВЛ;
- монитор;
- пылесос;
- дыхание;
- часы;
- шаги;
- далёкий поезд;
- советский двор;
- электрический гул;

- ветер.

22. Spatial Audio

Для desktop:

- Web Audio API;
- positional audio;
- stereo panning;
- distance attenuation;
- low-pass filters;
- reverb zones.

Например:

Игрок слышит ИВЛ слева.

Идёт влево.

Звук становится громче.

Поворачивается.

Аппарат оказывается за стеной.

23. Responsive UX

Целевые классы:

320–479 px
480–767 px
768–1023 px
1024–1439 px
1440–1919 px
1920+
Ultrawide

Поддержать:

- portrait;
- landscape;
- 16:9;
- 16:10;
- 4:3;
- 21:9;
- динамический viewport;
- safe-area insets.

Не использовать фиксированную игровую камеру, рассчитанную только на 16:9.

24. Mobile

Управление:

- touch;
- virtual joystick;

- tap;
- swipe;
- hold.

Дополнительно:

- vibration;
- device orientation, если разрешено;
- fullscreen;
- installable PWA.

Вибрация должна иметь fallback и не быть обязательной.

25. PWA

Обязательные компоненты:

manifest.webmanifest
service-worker
offline shell
icons
splash configuration
install prompt
cache strategy

Кэшировать:

- engine;
- UI;
- common textures;
- common audio;
- common shaders;
- low-resolution scare videos.

Большие видео и редкие ассеты можно загружать по необходимости.

26. Производительность

Целевые значения:

Desktop

60 FPS.

Mobile

30 FPS minimum.

60 FPS --- целевой режим на современных устройствах.

Использовать:

- instancing;
- object pooling;
- texture atlases;
- LOD;

- frustum culling;
- occlusion where practical;
- compressed textures;
- lazy loading;
- asset streaming;
- adaptive resolution;
- dynamic effects quality.

27.1 Автоматическая генерация видео и публикация

Начиная с версии 1.1, архитектура проекта фиксирует принцип:

ComfyUI не является обязательной частью 37.2 Dream и не входит в основной production pipeline.

Основным источником визуального контента является собственный procedural Dream Engine на TypeScript + Three.js. Это позволяет использовать один и тот же DreamSeed для:

- интерактивного browser experience;
- воспроизводимого cinematic render;
- 5-минутного YouTube-эпизода;
- отдельного 37-секундного Short;
- thumbnail;
- метаданных и Dream Journal.

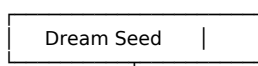
LLM используется как Dream Director, а не как генератор кадров. Он может формировать высокоуровневую драматургию, реплики, narration, camera mood, sound direction, название и описание, но не должен самостоятельно определять физику или нарушать deterministic core.

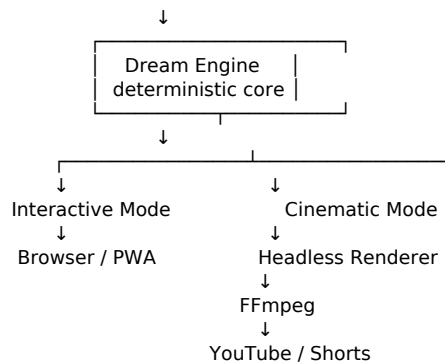
Production pipeline



Главный принцип

Browser Game и Video Factory используют один Dream Engine, а не две независимые системы генерации.





Это гарантирует, что сон, который пользователь увидел в браузере, можно воспроизвести как конкретный видеоэпизод по тому же seed.

27.2 Cinematic Render Mode

В Three.js необходимо предусмотреть отдельный режим:

Cinematic Render Mode

Он не зависит от пользовательского FPS и не должен захватывать видео простым screen recording.

Рендер должен управляться детерминированным simulation clock:

```

frame 0
↓
simulation time = 0.000
↓
update world
↓
render frame
↓
frame 1
↓
simulation time = 0.0333
↓
...
  
```

Для 30 FPS:

$dt = 1 / 30$

Для 60 FPS:

$dt = 1 / 60$

Таким образом один и тот же seed и одинаковые параметры рендера должны давать одинаковую последовательность кадров.

Не рекомендуется делать production video pipeline зависимым от:

- реального времени;
- текущего FPS браузера;
- пользовательского ввода;
- случайного `Math.random()`;
- screen capture рабочего стола.

27.3 Headless Rendering

Для автоматической генерации эпизодов используется headless browser или специализированный offscreen WebGL/WebGPU renderer.

Предпочтительная схема MVP:

```

.NET Render Worker
↓
  
```

Chromium / Headless Browser
↓
37.2 Dream Web Renderer
↓
Frame sequence
↓
FFmpeg

Renderer получает:

- DreamSeed;
- DreamProfile;
- SceneGraph;
- cinematic timeline;
- camera path;
- resolution;
- FPS;
- quality preset.

Renderer возвращает:

- последовательность кадров либо video stream;
- render manifest;
- timing information;
- ошибки/метрики;
- seed и renderer version.

Важно сохранять `rendererVersion`, поскольку изменение движка в будущем может изменить внешний вид старого seed.

27.4 Cinematic Director

Над Scene Graph располагается слой постановки:

Dream Scene
↓
Cinematic Director

- Shot Selection
- Camera Path
- Camera Lens
- Character Blocking
- Lighting
- Focus / DOF
- Transition
- Timing
- Scare Timing

Камера должна поддерживать несколько типов:

- cockpit / first person;
- rear / camera behind character;
- cinematic follow;
- wide establishing shot;
- top-down;
- track/scene overview;
- close-up;
- impossible camera;

- falling camera.

Камера является частью seed-driven timeline и должна быть воспроизводимой.

27.5 Audio Production Pipeline

Звук также должен быть частью deterministic timeline.

Базовые категории:

Ambient
Location
Physics
Recurring
Tension
Scare
Transition
Narration

Вместо записи звука непосредственно во время browser render создаётся audio/event manifest:

```
interface AudioCue {
  id: string;
  startTime: number;
  duration?: number;
  asset: string;
  volume: number;
  pitch?: number;
  spatial?: boolean;
  category: string;
}
```

После этого аудио собирается в финальный master через FFmpeg и/или offline audio rendering.

Ключевые фирменные звуки проекта:

- скрип качелей;
- ИВЛ;
- рір... рір...;
- пылесос;
- гул турбины;
- советский двор;
- шаги;
- дыхание;
- тиканье;
- звук падения;
- внезапная тишина.

27.6 LLM как Dream Director

LLM не является ядром генерации мира.

Его роль:

```
Dream Engine
↓
Structured Scene Graph
↓
LLM Dream Director
↓
Cinematic / Narrative Instructions
```

LLM может генерировать:

- краткую драматургическую дугу;

- реплики;
- narration;
- визуальное описание конкретной сцены;
- camera mood;
- sound direction;
- название;
- description;
- subtitles;
- teaser/hook для Short.

LLM не должен самостоятельно:

- менять seed;
- создавать физические правила;
- обходить compatibility matrix;
- бесконтрольно добавлять новые объекты;
- ломать continuity;
- определять случайность runtime.

Это позволяет сохранить авторский стиль и воспроизводимость.

27.7 5-минутный эпизод и 37-секундный Short

Каждый Dream Seed может порождать два разных cinematic products.

Full Episode

Целевая длительность:

≈ 5 минут

Формат:

16:9
1080p MVP
30 FPS MVP

Эпизод строится как самостоятельный сон:

```

Hook
↓
Normality
↓
First anomaly
↓
Escalation
↓
Absurd peak
↓
False calm
↓
Scare / punchline
↓
Awakening

```

Short

Short не должен быть простым crop/clip из длинного видео.

Он имеет отдельную постановку:

0-3s Hook
3-12s Setup
12-25s Escalation
25-32s WTF moment
32-35s False calm
35-37s Scare / punchline

Формат:

9:16
37 seconds exactly

Short использует тот же Dream Seed, но может иметь отдельный ShortSeed:

ShortSeed = Hash(DreamSeed + "SHORT")

Это позволяет сохранить связь с основным сном и одновременно сделать Short самостоятельным произведением.

27.8 Thumbnail и metadata factory

После рендера автоматически создаются:

- thumbnail;
- title;
- description;
- hashtags;
- subtitles;
- episode ID;
- Dream Seed reference.

Thumbnail желательно получать из реального кадра эпизода, а не создавать полностью отдельной системой, чтобы визуальная идентичность совпадала с видео.

Для каждого опубликованного видео сохраняются:

DreamSeed
RendererVersion
EngineVersion
EpisodeVersion
ShortVersion
ThumbnailVersion
MetadataVersion

27.9 Автоматический production loop

Production factory может работать пакетно:

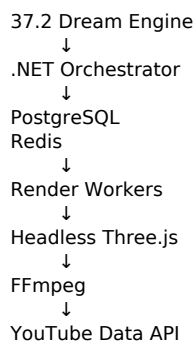
```
Generate N seeds
↓
Validate uniqueness
↓
Build Dream Profiles
↓
Build Scene Graphs
↓
Director pass
↓
Render
↓
Quality checks
↓
FFmpeg encode
↓
Thumbnail / metadata
↓
Upload / schedule
```

Перед публикацией необходимо выполнять автоматические проверки:

- render completed;
- audio exists;
- no missing assets;
- expected duration;
- expected aspect ratio;
- expected FPS;
- no broken scene;
- no duplicate recent dream;
- no empty/black render beyond configured limits;
- subtitles match final timeline;
- metadata generated.

27.10 .NET Orchestrator

Для production automation рекомендуется отдельный C#/.NET orchestrator.



PostgreSQL

Хранит:

- Dreams;
- Seeds;
- Dream Profiles;
- Scene Graphs;
- Episodes;
- Shorts;
- Render Jobs;
- Render Versions;
- publication metadata;
- generation history.

Redis

Используется для:

- locks;
- job state;

- distributed coordination;
- rate limiting;
- temporary render state;
- caching.

Kafka

Не обязателен для MVP.

Подключается при появлении реальной потребности в распределённой production pipeline:

```
DreamGenerated
RenderQueued
RenderCompleted
VideoEncoded
ThumbnailGenerated
UploadCompleted
PublishScheduled
```

27.11 Renderer abstraction

Чтобы не закрывать путь к AI-video в будущем, renderer должен иметь абстракцию:

```
interface IRenderProvider {
  render(request: RenderRequest): Promise<RenderResult>;
}
```

Первичная реализация:

```
ThreeJsRenderer
```

Возможная будущая реализация:

```
RemoteAiVideoRenderer
```

Это означает:

- MVP работает без ComfyUI;
- основной контент остаётся deterministic;
- отдельные сцены в будущем можно усиливать AI-video;
- Dream Engine не зависит от конкретного видеогенератора.

ComfyUI может быть подключён позднее как внешний экспериментальный provider, но не должен становиться обязательной зависимостью проекта.

27.12 Стратегия разработки

Перед созданием полной video factory необходимо сделать вертикальный срез:

```
1 DreamSeed
↓
Dream Profile
↓
Scene Graph
↓
30-60 sec cinematic scene
↓
Headless render
↓
FFmpeg
↓
готовый Short
```

После подтверждения качества:

```
30-60 sec
↓
```

5 min Episode
↓
37 sec Short
↓
Batch rendering
↓
Automatic publishing

Это снижает риск построить сложную инфраструктуру до проверки художественного качества.

27.13 Почему ComfyUI не является базовым решением

Для проекта 37.2 Dream важнее:

- стабильность персонажей и объектов;
- deterministic seed;
- физика;
- повторяемость;
- процедурные комбинации;
- авторская логика;
- единый игровой и cinematic world;
- контролируемые переходы;
- recurring characters;
- повторяющиеся звуковые мотивы.

Генеративное видео может дать более «нейросетевой» визуальный wow-effect, но одновременно повышает риск:

- визуального дрейфа;
- неповторяемости;
- нарушения continuity;
- усложнения автоматизации;
- GPU-зависимости;
- роста стоимости production.

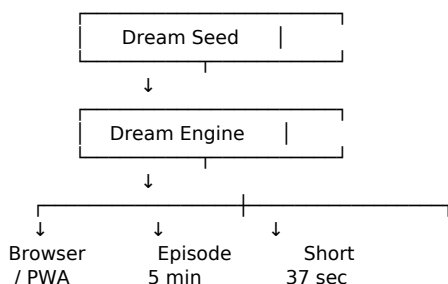
Поэтому базовый production path версии 1.1:

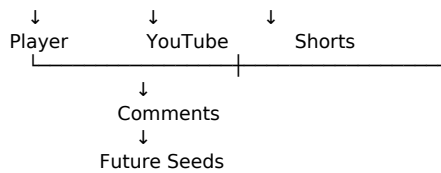
Procedural 3D + Cinematic Director + Headless Renderer + FFmpeg + LLM Director + YouTube API.

27.14 Единый контентный цикл проекта

В версии 1.1 37.2 Dream рассматривается не только как browser game.

Полная система:





Таким образом YouTube становится не отдельным проектом, а distribution layer поверх того же Dream Engine.

27. Рекомендуемый технологический стек

Client

TypeScript

WebGL/WebGPU-compatible rendering layer

Для MVP предпочтительно:

- Three.js;
- TypeScript;
- Web Audio API;
- Vite;
- PWA plugin.

WebGPU можно добавить как progressive enhancement, сохранив fallback.

Backend

MVP может работать без backend.

Local-first:

- seed;
- история снов;
- настройки;
- journal;

хранятся локально.

Позже можно добавить backend для:

- sharing dreams;
- global statistics;
- dream gallery;
- leaderboards;
- generated dream IDs.

28. Suggested project structure

```

37-2-dream/
├── apps/
│   └── web/
├── packages/
│   ├── dream-core/
│   ├── dream-generator/
│   ├── scene-system/
│   ├── physics/
│   └── audio/

```

- | — scares/
- | — procedural-world/
- | — ui/
- | — shared/
- | — assets/
 - | — audio/
 - | — textures/
 - | — models/
 - | — videos/
 - | — fonts/
- | — docs/
- | — tests/
- | — tools/
 - | — renderer/
 - | — ffmpeg/
 - | — youtube/
 - | — generation/

29. Основные интерфейсы

```
interface DreamSeed {
  value: string;
}

interface DreamProfile {
  temperature: number;
  absurdity: number;
  anxiety: number;
  medicalIntensity: number;
  sovietIntensity: number;
  domesticIntensity: number;
  physicsInstability: number;
  jumpscareIntensity: number;
  dreamLength: number;
}

interface DreamScene {
  id: string;
  type: string;
  seed: string;
  duration: number;
  intensity: number;
  tags: string[];
}

interface DreamTransition {
  from: string;
  to: string;
  type: string;
  probability: number;
}

interface DreamEvent {
  id: string;
  type: string;
  trigger: string;
  cooldown: number;
  rarity: number;
}

interface DreamHistory {
  scenes: string[];
  transitions: string[];
  scares: string[];
  recurringEvents: string[];
}
```

30. Алгоритм генерации

Псевдокод:

```
function generateDream(seed: string): Dream {
  const rng = createSeededRng(seed);

  const profile = generateProfile(rng);

  const history = loadDreamHistory();
```

```

const sceneCount = calculateSceneCount(profile, rng);

const scenes = [];

for (let i = 0; i < sceneCount; i++) {
  const candidates = getCompatibleScenes(
    scenes.at(-1),
    profile
  );

  const filtered = applyCooldowns(
    candidates,
    history
  );

  const scene = weightedPick(
    filtered,
    rng
  );

  scenes.push(
    generateScene(scene, profile, rng, history)
  );
}

return connectScenes(scenes, profile, rng);
}

```

31. Генерация событий внутри сцены

Сцена не должна быть статичной.

Например:

```

0-20%  normal
20-40% strange
40-60% absurd
60-75% tension
75-90% event
90-100% transition

```

Но эти интервалы тоже должны рандомизироваться.

32. Recurring characters

Минимальный набор:

Женщина с пылесосом

Не объясняется.

Голубь ЖКХ

Может давать бессмысленные комментарии.

Мужчина в автобусе

Всегда знает, где выход.

Но выхода нет.

Сам игрок

Может появляться в качестве спящего двойника.

33. Dream Echo

Увиденный объект может вернуться в другом контексте.

Например:

1. Игрок видит качели.
2. Потом слышит их скрип в больнице.
3. Потом видит качели внутри автобуса.
4. Потом качели пролетают во время падения.
5. В финале качели стоят рядом с кроватью.

Это должно создавать ощущение, что сон имеет внутреннюю память.

34. Dream Journal

После сна:

СОН №1847

Температура: 37.2°C

Длительность: 08:43

Локации:

- советский двор
- лестница
- варенье
- больница

События:

- 4 падения
- 2 скриммера
- 1 голубь ЖКХ
- 3 появления пылесоса
- 7 термометров

Самый странный объект:

Лифт с рыбами

Причина пробуждения:

Слишком высокая нестабильность сна

35. Причины пробуждения

Варианты:

- температура достигла критического значения;
- слишком сильный скриммер;
- случайный переход;
- столкновение;
- падение;
- сон закончился;
- «мозг решил, что хватит»;
- случайное пробуждение.

Последний вариант может быть представлен абсолютно сухим текстом:

ПРИЧИНА: НЕИЗВЕСТНО.

36. UX-принцип

Не объяснять всё.

Никаких длинных tutorial screens.

Игрок должен постепенно понять:

- 37,2 --- центральный мотив;
- сон имеет память;
- объекты возвращаются;
- мир процедурный;
- физика ненадёжна;
- скриммеры могут быть ложными;
- звуки могут предсказывать события;
- качели являются одним из ключевых переходов.

37. MVP

Первый playable prototype:

6 локаций

- советский двор;
- лестница;
- квартира;
- падение;
- варенье;
- термометры.

5 recurring events

- качели;
- пылесос;
- ИВЛ;
- монитор;
- голубь.

3 перехода

- падение;
- дверь;
- изменение мира.

2 скриммера

- false scare;
- real scare.

1 генератор

Seed → profile → scene graph → runtime.

38. Phase 2

Добавить:

- автобус;
- лифт;
- пустыню;
- больницу;
- море;
- дополнительные recurring characters;
- видео-скриммеры;
- vibration;
- spatial audio;
- dream journal;
- replay по seed.

39. Phase 3

Добавить:

- community dream seeds;
- sharing;
- QR-код конкретного сна;
- URL вида:
/ dream / 37-2-A81F9C
- просмотр сна другого пользователя;
- ежедневный seed;
- коллекцию редких событий;
- скрытые сцены.

40. Репозиторий

Рекомендуемое название:

37-2-dream

GitHub URL:

`github.com/<your-user>/37-2-dream`

Название короткое, запоминающееся и сразу связано с центральной механикой проекта.

Альтернативы:

- dream-37-2
- fever-dream-37-2
- 37-2-fever-dream
- dream-at-37-2

- 37-2-sleep
- fever-dream-machine

Основное название проекта:

37.2 Dream

Подзаголовок:

A procedurally generated fever dream.

41. Ключевой принцип проекта

Проект не должен стремиться к бесконечному количеству контента.

Его сила --- в комбинаторике.

Например:

- 20 locations
- × 30 events
- × 15 transitions
- × 10 physics modifiers
- × 20 audio events
- × 15 scare events
- × procedural parameters

дают огромное число возможных последовательностей при относительно небольшом количестве handcrafted assets.

Главная задача разработчика --- не создавать тысячи сцен, а создавать много хорошо сочетающихся элементов.

42. Definition of Done

MVP считается готовым, если:

- каждый новый seed создаёт отличающийся маршрут;
- одинаковый seed воспроизводим;
- сцены не повторяются подряд без специальной причины;
- recurring jokes имеют cooldown;
- переходы процедурные;
- физика может изменяться между сценами;
- аудио реагирует на состояние сна;
- ИВЛ и монитор появляются контекстно;
- есть false и real scares;
- есть хотя бы один видео-скример;
- есть советский двор с качелями;
- есть падение;
- есть варенье;
- есть лестницы без перил;
- есть 37,2 °C как глобальный мотив;
- игра работает в desktop и mobile;

- PWA устанавливается;
- игра сохраняет seed и журнал локально;
- при потере сети уже загруженный основной контент продолжает работать.

42.1 Definition of Done для Video Factory v1.1

Video Factory считается готовой для MVP, если:

- один DreamSeed воспроизводимо создаёт одинаковый Scene Graph;
- browser mode и cinematic mode используют один Dream Engine;
- cinematic render работает без screen recording;
- render управляется deterministic simulation clock;
- headless renderer способен получить минимум 30–60 секунд готового видео;
- FFmpeg собирает корректный video + audio master;
- генерируется отдельный 37-секундный Short;
- Short имеет вертикальный формат 9:16;
- full episode имеет горизонтальный формат 16:9;
- сохраняются EngineVersion и RendererVersion;
- отсутствует обязательная зависимость от ComfyUI;
- LLM не управляет физикой и deterministic randomness;
- metadata и thumbnail могут быть сгенерированы автоматически;
- pipeline умеет выполнить end-to-end тест Seed → Render → Encode;
- предусмотрен интерфейс IRendererProvider для будущего AI renderer;
- YouTube publishing layer изолирован от Dream Engine.

43. Итоговое видение

37.2 Dream --- это не просто генератор случайных сюрреалистических сцен.

Это система, которая создаёт ощущение:

«Я понятия не имею, почему я здесь, но почему-то всё это кажется совершенно нормальным».

Игрок должен выйти из каждого сна с ощущением, что он действительно побывал внутри чужого, больного, нелогичного, но удивительно последовательного сна.

Главный художественный контраст:

абсолютно серьёзная атмосфера + максимально идиотские события.

И центральная фраза проекта:

37,2 °C. Ничего страшного. Просто приснилось.